

Questions on Unit 04 Sections 07-09

The following multiple-choice questions are designed to assess your knowledge of this unit's section. See the version without solutions first. Each question is worth 1.0 points, with the score divided among the correct answers, if applicable. You should aim to complete each question within an average time of three to four minutes. You may solve the questions in any order you prefer; once finished, compare your responses with the provided solution file. You have successfully passed the assessment if you achieve at least 50% of the available points.

Ex. Static vs. stack vs. heap memory

Choose one or more options

- ☒ Static memory stores objects defined outside any functions and static class data members; their lifetime depends on their scope and is managed automatically by the compiler.
- ☒ Stack memory stores non-static local objects defined inside functions; they are automatically managed by the compiler.
- ☒ Heap memory stores dynamic objects allocated manually (e.g., with `new` in C++ or `malloc` in C); the programmer controls allocation and deallocation.
- ☐ All C++ containers (e.g., `std::vector`) store their internal data structures on the stack, never on the heap.
- ☒ Even if a container object itself is declared on the stack, its internal buffer (where the actual data is stored) is usually allocated on the heap.

Ex. C-style dynamic memory (malloc/free) vs. C++ (new/delete)

Choose one or more options

- ☐ `malloc` and `free` automatically call constructors and destructors for objects, just like `new` and `delete`.
- ☒ `malloc` and `free` are library functions that return raw memory (`void*`), requiring explicit type casting and manual initialization.
- ☒ `new` and `delete` are operators in C++ that return fully typed pointers and call constructors/destructors automatically.
- ☒ `new` can be overloaded to customize memory allocation behavior; `malloc` cannot be overloaded.
- ☒ On failure, `new` throws an exception (`std::bad_alloc`), while `malloc` returns `NULL` (no exceptions).

Ex. Using `new` to allocate single variables. Given the following code snippet:

```
int *v1 = new int;  
int *v2 = new int();  
int *v3 = new int(12);
```

Choose one or more options

- ☒ `v1` points to a single `int` with undefined value (similar to `malloc`).
- ☒ `v2` points to a single `int` initialized to 0 (similar to `calloc`).
- ☒ `v3` points to a single `int` initialized to 12.
- ☐ All three allocations (`v1`, `v2`, `v3`) automatically initialize the integer to 0.
- ☐ If any of these allocations fail, `new` will return a `nullptr` by default (without throwing an exception).

Ex. `new` with arrays and exceptions. Given the following code snippet:

```
int *vect1 = new int[10];
int *vect2 = new (nothrow) int[20];
```

Choose one or more options

- ☒ vect1 is an array of 10 integers; if allocation fails, new throws std::bad_alloc.
- ☒ vect2 is an array of 20 integers; if allocation fails, new (nothrow) returns nullptr and does not throw an exception.
- ☐ Both allocations always return nullptr on failure and never throw exceptions.
- ☒ The normal version of new never returns nullptr; it always throws an exception on failure.
- ☐ The nothrow version of new always throws std::bad_alloc if memory allocation fails.

Ex. Allocating constant objects with new. Given the following code snippet:

```
const int *p4 = new const int(100);
const my_class *p5 = new const my_class;
```

Choose one or more options

- ☒ p4 points to a dynamically allocated constant integer initialized to 100.
- ☒ p5 points to a dynamically allocated constant my_class object initialized via the default constructor.
- ☒ Constant objects allocated with new must be initialized (explicitly or implicitly via a constructor).
- ☐ The pointers p4 and p5 themselves are const, preventing reassignment of the pointer.
- ☐ It is illegal to allocate const objects with new in C++.

Ex. Pointer indexing and bounds checking. Given the following code snippet:

```
double *p1 = new double;
*p1 = 7.3;
p1[0] = 8.2;
p1[7] = 9.4;    // Error
p1[-4] = 2.4;   // Error

double *p2 = new double[10];
*p2 = 7.3;
p2[0] = 8.2;
p2[7] = 9.4;
p2[-4] = 2.4;   // Error
```

Choose one or more options

- ☒ A pointer does not know the number of elements it points to; it only knows the address of the first element.
- ☒ For p1 (single double), accessing p1[7] or p1[-4] is undefined behavior (out-of-bounds).
- ☒ For p2 (array of 10), p2[7] is valid, but p2[-4] is undefined behavior.
- ☐ C++ automatically checks array bounds at runtime, so p1[7] will throw an exception.
- ☐ Pointers in C++ store metadata about the array size, allowing runtime bounds checking.

Ex. Pointer type safety. Given the following code snippet:

```
int *p1 = new int(10);
int *p2 = p1;
float *p3 = p1;    // Error
char *p4 = p1;      // Error
```

Choose one or more options

- ☒ A pointer does not know the actual type of object it points to; it just knows the address and assumes the declared type.
- ☒ `p2 = p1` is valid because both are `int*`.
- ☐ `p3 = p1` is valid because pointers can be freely converted between numeric types.
- ☐ `p4 = p1` is valid because `char*` can point to any memory location.
- ☒ Assigning an `int*` to a `float*` or `char*` without a cast is a type error in C++.

Ex. Using `delete` to free memory. Given the following code snippet:

```
int *v1 = new int;
int *v2 = new int(12);
int *vect1 = new int[10];
my_class *p = new my_class;

delete v1;
delete v2;
delete[] vect1;
delete p;
```

Choose one or more options

- ☒ `delete` frees memory allocated for a single object and calls its destructor.
- ☒ `delete[]` frees memory allocated for an array and calls the destructor for each element.
- ☒ For each call to `new`, there should be a corresponding call to `delete` (or `delete[]` for arrays).
- ☐ Using `delete` on an array (instead of `delete[]`) is safe and will correctly deallocate all elements.
- ☐ Using `delete[]` on a single object is safe and has the same effect as `delete`.

Ex. Lifetime of dynamically allocated objects. Given the following code snippet:

```
my_type *my_func(...arg) {
    return (new my_type(arg));
}

...
my_type *p = my_func(arg);
```

Choose one or more options

- ☒ The dynamically allocated object pointed to by `p` exists until it is explicitly deleted.
- ☒ If `p` goes out of scope, the pointer is destroyed, but the memory it points to is NOT automatically freed.
- ☐ The memory referenced by `p` is automatically freed when `p` goes out of scope.
- ☒ The caller of `my_func` is responsible for using the object and deleting `p` when done.
- ☐ The compiler automatically inserts a `delete` call when `p` goes out of scope, preventing memory leaks.

Ex. Shallow copy. Given the following code snippet:

```
class Box {
public:
    int *v;
    Box(int value) { v = new int(value); }
};
```

```
int main() {
    Box box1(10);
    Box box2 = box1;
    *box2.v = 20;
    std::cout << *box1.v;
    return 0;
}
```

Choose one or more options

- ☒ A shallow copy duplicates the pointer value (memory address), not the data it points to.
- ☒ After `Box box2 = box1;`, both `box1.v` and `box2.v` point to the same `int` in memory.
- ☒ Modifying `*box2.v` changes the value seen by `*box1.v`, so the output is 20.
- ☐ The default copy constructor performs a deep copy, so `box1.v` and `box2.v` point to different memory locations.
- ☐ The output of the program is 10, because each box has its own independent copy of `v`.

Ex. Deep copy. Given the following code snippet:

```
class Box {
public:
    int *v;
    Box(int value) { v = new int(value); }
    Box(const Box &tmp) { v = new int(*tmp.v); } // Deep copy constructor
};

int main() {
    Box box1(10);
    Box box2 = box1;
    *box2.v = 20;
    std::cout << *box1.v;
    return 0;
}
```

Choose one or more options

- ☒ A deep copy duplicates the object and all objects it refers to recursively.
- ☒ The user-defined copy constructor allocates new memory for `box2.v` and copies the value from `box1.v`.
- ☒ After the copy, `box1.v` and `box2.v` point to different memory locations, so the output is 10.
- ☐ Without the custom copy constructor, the default behavior would also perform a deep copy.
- ☐ Modifying `*box2.v` affects `*box1.v`, so the output is 20.

Ex. Dangling pointers

Choose one or more options

- ☒ A dangling pointer is one that points to memory that has been freed but still holds a valid memory address.
- ☒ Dangling pointers are generated when memory is released (e.g., via `delete`), but the pointer variable is not reset.
- ☒ Using a dangling pointer can cause undefined behavior, including memory violations and crashes.
- ☐ Dangling pointers automatically become `nullptr` after `delete` is called, preventing accidental access.

- ☒ Dangling pointers are very difficult to discover and trace back during debugging.

Ex. Memory leaks

Choose one or more options

- ☒ A memory leak occurs when dynamically allocated memory is not freed, and the program loses all references to it.
- ☒ If a function leaks 1 byte but is called 10^7 times, the program leaks ~10 MB total.
- ☐ Memory leaks are acceptable in most programs because modern operating systems reclaim leaked memory immediately.
- ☒ Many programs (especially long-running servers) cannot afford memory leaks, as they degrade performance over time.
- ☐ The C++ compiler automatically detects and prevents all memory leaks at compile time.

Ex. Setting pointers to `nullptr` to avoid dangling pointers. Given the following code snippet:

```
int *ptr;
delete ptr;

int *ptr = nullptr;
delete ptr;
ptr = nullptr;
```

Choose one or more options

- ☒ Setting a pointer to `nullptr` when defining it or after deleting it helps avoid dangling pointers.
- ☒ Calling `delete` on a `nullptr` is safe and does nothing.
- ☐ Setting `ptr = nullptr` after `delete ptr` completely eliminates the risk of memory leaks.
- ☐ If multiple pointers point to the same memory, setting one to `nullptr` after `delete` protects all of them from becoming dangling.
- ☒ Standard strategies like setting pointers to `nullptr` provide only limited protection against dangling pointers and do not prevent memory leaks.

Ex. Dangling pointers with multiple pointers. Given the following code snippet:

```
int *p = new int();
auto q = p;
...
delete p;
p = nullptr;
```

Choose one or more options

- ☒ After `delete p; p = nullptr;`, `p` is safe (points to `nullptr`), but `q` is still dangling.
- ☒ Resetting `p` has no effect on `q`, which still holds the address of the freed memory.
- ☐ Setting `p = nullptr` automatically updates all other pointers (`q`) pointing to the same memory.
- ☐ The compiler will issue a warning or error if `q` is used after `p` is deleted.
- ☒ It is hard to ensure that we free memory at the right time when multiple pointers share ownership.

Ex. Early return and memory leaks. Given the following code snippet:

```
void foo(unsigned length) {
    int* buffer = new int[length];
    ...
    if (condition)
```

```

    return;
...
delete[] buffer;
return;
}

```

Choose one or more options

- ☒ If condition is true, the function returns early without calling `delete[] buffer`, causing a memory leak.
- ☐ The compiler automatically inserts `delete[] buffer` before any return statement, preventing leaks.
- ☒ It is hard to ensure that memory is freed at the right time when there are multiple exit points in a function.
- ☐ The operating system will reclaim `buffer` when the function returns, so no leak occurs.
- ☒ To avoid this leak, you should use RAII (e.g., smart pointers) or ensure `delete[] buffer` is called on all exit paths.

Ex. RAII (Resource Acquisition Is Initialization)

Choose one or more options

- ☒ RAII binds the lifetime of a resource to the lifetime of an object (e.g., a smart pointer).
- ☒ The resource is acquired in the constructor and released in the destructor, ensuring automatic cleanup.
- ☒ RAII eliminates redundant tests and memory leaks by tying resource management to object scope.
- ☐ RAII requires manual calls to `delete` or `free` to release resources when they are no longer needed.
- ☒ Smart pointers (`shared_ptr`, `unique_ptr`, `weak_ptr`) implement RAII to manage dynamic memory automatically.

Ex. Smart pointers overview

Choose one or more options

- ☒ C++ offers three types of smart pointers: `shared_ptr`, `unique_ptr`, and `weak_ptr`.
- ☒ Smart pointers are templates and work with all types.
- ☒ `shared_ptr` allows multiple pointers to share ownership of the same object via reference counting.
- ☒ `unique_ptr` represents exclusive ownership; only one `unique_ptr` can own a resource at a time.
- ☐ `weak_ptr` owns the object it points to and automatically deletes it when the `weak_ptr` goes out of scope.

Ex. `shared_ptr` and reference counting

Choose one or more options

- ☒ `shared_ptr` uses a reference count to track how many pointers share ownership of a resource.
- ☒ The reference count is incremented when we copy the `shared_ptr`.
- ☒ The reference count is decremented when a `shared_ptr` goes out of scope or is assigned a new value.
- ☒ When the reference count goes to zero, the resource is automatically deleted.
- ☐ `shared_ptr` does not use reference counting; it relies on garbage collection instead.

Ex. Allocating `shared_ptr` with `new` (NOT recommended). Given the following code snippet:

```
shared_ptr<int> p1 = new int(42);           // Error
shared_ptr<int> p2(new int(42));           // OK
shared_ptr<int> p3 = shared_ptr<int>(new int(9)); // OK
```

Choose one or more options

- ☐ `p1` is correct because implicit conversion from raw pointer to `shared_ptr` is allowed.
- ☒ `p1` is an error because `shared_ptr` does not allow implicit conversion from a raw pointer.
- ☒ `p2` is correct (direct initialization with raw pointer).
- ☒ `p3` is correct (explicit conversion from raw pointer).
- ☒ Using `new` with `shared_ptr` is inefficient because it requires two separate memory allocations (one for the object, one for the control block).

Ex. `make_shared` (recommended).

Choose one or more options

- ☒ `make_shared<T>(args)` allocates and initializes an object of type `T` and returns a `shared_ptr` pointing to it.
- ☒ `make_shared` performs a single memory allocation that includes both the object and its control block (reference counter).
- ☐ `make_shared` is slower than using `new` with `shared_ptr` because it performs two allocations.
- ☒ Using `make_shared` is the preferred way to create `shared_ptr` objects.
- ☐ `make_shared` does not initialize the object; it only allocates memory.

Ex. `unique_ptr` allocation. Given the following code snippet:

```
std::unique_ptr<int> valuePtr(new int(15));
```

Choose one or more options

- ☒ `unique_ptr` represents exclusive ownership of a dynamically allocated object.
- ☒ `unique_ptr` does not have a `make_unique` function (in C++11), so we must use `new`.
- ☐ `unique_ptr` allows multiple pointers to share ownership of the same resource.
- ☒ When `valuePtr` goes out of scope, the destructor automatically deletes the allocated memory.
- ☐ `unique_ptr` requires manual calls to `delete` to free memory.

Ex. `unique_ptr` and no memory leaks. Given the following code snippet:

```
void my_func() {
    std::unique_ptr<int> valuePtr(new int(15));
    ...
    if (...)
        return;
    ...
}
```

Choose one or more options

- ☒ There is no memory leak, even if the function returns early via `if (...)`.
- ☒ The `unique_ptr` destructor automatically deletes the allocated memory when it goes out of scope.
- ☐ The programmer must manually call `delete` on `valuePtr` before each `return` to avoid a leak.

- ☒ Using `unique_ptr` implements RAI, ensuring resources are released automatically.
- ☐ `unique_ptr` does not call the destructor if the function exits early.

Ex. Returning `unique_ptr` from a function. Given the following code snippet:

```
unique_ptr<int> clone1(int p) {
    return unique_ptr<int>(new int(p));
}
```

```
unique_ptr<int> clone2(int p) {
    unique_ptr<int> lp(new int(p));
    return lp;
}
```

Choose one or more options

- ☒ We cannot copy a `unique_ptr`, but we can return a local `unique_ptr` from a function (move semantics apply).
- ☒ `clone1` directly returns a newly created `unique_ptr`.
- ☒ `clone2` returns a local `unique_ptr` variable; the compiler uses move semantics (or RVO) to transfer ownership.
- ☐ Returning a local `unique_ptr` causes a compilation error because copying is disallowed.
- ☐ Both functions cause memory leaks because `unique_ptr` cannot be returned from a function.

Ex. `weak_ptr` overview

Choose one or more options

- ☒ `weak_ptr` is a smart pointer that does not control the lifetime of the object it points to.
- ☒ `weak_ptr` points to an object managed by a `shared_ptr` without increasing the reference count.
- ☐ `weak_ptr` owns the object it points to and automatically deletes it when the `weak_ptr` goes out of scope.
- ☒ To use a `weak_ptr`, we must check if it is still valid using `expired()` and `lock()`.
- ☒ Creating a `weak_ptr` from a `shared_ptr` does not change the reference count of the `shared_ptr`.

Ex. Using `weak_ptr`. Given the following code snippet:

```
struct person { string name; person(string n):name(n){} };
```

```
shared_ptr<person> p1 = make_shared<person>("Jack");
shared_ptr<person> p2;
shared_ptr<person> p3;
```

```
p2 = p1;
weak_ptr<person> wp(p1);
```

```
if (p3 = wp.lock()) {
    cout << p3->name << endl; // Jack
}
```

```
p1.reset(new person("rose"));
p2.reset();
p3.reset();
```

Choose one or more options

- ☒ `wp` is a `weak_ptr` initialized from `p1`; it does not increase the reference count.

- ☒ `wp.lock()` returns a `shared_ptr` if the object still exists; otherwise, it returns a null `shared_ptr`.
- ☒ After `p1.reset(new person("rose")); p2.reset(); p3.reset();`, the original "Jack" object is deleted (if no other `shared_ptr` owns it).
- ☐ `wp` owns the "Jack" object, preventing it from being deleted when `p1`, `p2`, and `p3` are reset.
- ☐ Calling `wp.lock()` will always return a valid `shared_ptr`, even after all `shared_ptr` instances are reset.

Ex. Copy constructor overview. Given the following code snippet:

```
class Foo {
public:
    Foo();                // Primary constructor
    Foo(const Foo&);       // Copy constructor
};
```

Choose one or more options

- ☒ A copy constructor creates a new object as a copy of an existing object of the same class.
- ☒ The copy constructor has the same name as the class and takes a reference to a `const` object of the same type.
- ☐ The copy constructor is called only when explicitly invoked, never implicitly.
- ☒ The copy constructor may have additional parameters with default values.
- ☐ The copy constructor is automatically deleted if the class has a destructor.

Ex. When is the copy constructor called? Given the following code snippet:

```
MyClass obj2 = obj1;
void foo(MyClass obj);
foo(obj1);
MyClass foo() { MyClass temp; return temp; }
MyClass my = foo();
```

Choose one or more options

- ☒ The copy constructor is called when creating a new object using an existing object (`MyClass obj2 = obj1;`).
- ☒ The copy constructor is called when an object is passed by value to a function (`foo(obj1);`).
- ☒ The copy constructor may be called when an object is returned from a function by value (unless RVO/NRVO is applied).
- ☐ The copy constructor is never called when returning an object from a function.
- ☐ The copy constructor is called when an object is passed by reference to a function.

Ex. Synthesized copy constructor (shallow copy). Given the following code snippet:

```
class Class {
public:
    Class(const char *str);
    ~Class();
private:
    char *str;
};

Class::Class(const char *s) {
    str = new char[strlen(s)+1];
    strcpy(str, s);
}
```

```

Class::~~Class() {
    delete[] str;
}

// Compiler-generated copy constructor:
Class::Class(const Class &another) {
    str = another.str;
}

```

Choose one or more options

- ☒ The synthesized copy constructor copies each non-static member from the given object to the created object.
- ☒ In this case, the synthesized copy constructor copies the pointer `str`, not the string data (shallow copy).
- ☐ The synthesized copy constructor automatically performs a deep copy of dynamically allocated memory.
- ☒ If we rely on the synthesized copy constructor for this class, we may encounter double-delete errors when both objects are destroyed.
- ☐ The synthesized copy constructor duplicates the string pointed to by `str`, allocating new memory.

Ex. User-defined copy constructor (deep copy). Given the following code snippet:

```

Class::Class(const Class &another) {
    str = new char[strlen(another.str)+1];
    strcpy(str, another.str);
}

```

Choose one or more options

- ☒ This user-defined copy constructor allocates new memory for `str` and copies the string data (deep copy).
- ☒ After the copy, each object has its own independent copy of the string, avoiding double-delete errors.
- ☐ This copy constructor performs a shallow copy, copying only the pointer value.
- ☐ User-defined copy constructors are never necessary; the compiler always generates the correct behavior.
- ☒ Defining a custom copy constructor is necessary when the class manages resources (e.g., dynamic memory, file handles).

Ex. Direct vs. copy initialization. Given the following code snippet:

```

// Direct initialization
string s1(10, '.');
string s2(s1);

// Copy initialization
string s3 = s1;
string s4 = "1234567890";
string s5 = string(100, '9');
string s6;

```

Choose one or more options

- ☒ Direct initialization calls the primary constructor that best matches the arguments.
- ☒ Copy initialization calls the copy constructor to copy the right-hand operand into the object being created.

- ☐ `string s2(s1);` uses copy initialization, not direct initialization.
- ☒ `string s3 = s1;` uses copy initialization.
- ☐ `string s6;` uses copy initialization to create an empty string.

Ex. Copy assignment operator overview. Given the following code snippet:

```
class Foo {
public:
    Foo();                      // Primary constructor
    Foo(const Foo&);            // Copy constructor
    Foo& operator=(const Foo&); // Copy assignment operator
};
```

Choose one or more options

- ☒ The copy assignment operator is used to assign the contents of one object to another object of the same class when both objects already exist.
- ☒ The copy assignment operator has the name `operator=` and takes a reference to a `const` object of the same type.
- ☒ The copy assignment operator typically returns a reference to `*this` to allow chaining (e.g., `a = b = c;`).
- ☐ The copy assignment operator is called when creating a new object from an existing object.
- ☐ The copy assignment operator is never automatically generated by the compiler.

Ex. When is the copy assignment operator called? Given the following code snippet:

```
obj2 = obj1;
MyClass foo() { MyClass temp; return temp; }
MyClass my1 = foo();
my2 = my1;
```

Choose one or more options

- ☒ The copy assignment operator is called when you use `=` to assign one object to another and both objects already exist (`obj2 = obj1;`).
- ☒ `MyClass my1 = foo();` may use the copy constructor (or move constructor), not the copy assignment operator.
- ☒ `my2 = my1;` calls the copy assignment operator if `my2` already exists.
- ☐ The copy assignment operator is called when a function returns an object by value.
- ☐ The copy assignment operator is called when creating a new object with `MyClass my1 = foo();`.

Ex. Synthesized copy assignment operator. Given the following code snippet:

```
class sales {
public:
    sales(const sales&);
    sales& operator=(const sales&);
private:
    std::string number;
    int sold = 0;
    double revenue = 0.0;
};

sales& sales::operator=(const sales &orig) {
    number = orig.number;
    sold = orig.sold;
    revenue = orig.revenue;
}
```

```
    return *this;
}
```

Choose one or more options

- ☒ The synthesized copy assignment operator assigns each non-static member from the source object to the target object.
- ☒ The user-defined copy assignment operator shown is equivalent to the synthesized version for this class.
- ☐ The synthesized copy assignment operator performs a deep copy of dynamically allocated memory.
- ☒ The copy assignment operator typically returns `*this` to enable assignment chaining.
- ☐ The compiler never generates a copy assignment operator automatically.

Ex. The Rule of Three

Choose one or more options

- ☒ If you define any of the destructor, copy constructor, or copy assignment operator, you should define all three.
- ☒ The Rule of Three ensures that your class correctly manages resources (e.g., memory) when objects are copied, assigned, or destroyed.
- ☐ The Rule of Three states that you must define at least three member functions in every class.
- ☒ If you define one but not the others, you might encounter problems such as memory leaks or double deletes.
- ☐ The Rule of Three is obsolete in modern C++ and has been replaced by the Rule of Zero.

Ex. Move semantics motivation

Choose one or more options

- ☒ Move semantics allow "moving" instead of copying resources when the source object is about to be destroyed.
- ☒ Moving can enhance performance by avoiding unnecessary copying of resource-heavy objects.
- ☐ Move semantics were introduced in C++98 to improve compatibility with C.
- ☒ Move operators typically "steal" resources and do not usually allocate new resources or throw exceptions.
- ☐ Move semantics are only applicable to primitive types like `int` and `double`.

Ex. Move constructor overview. Given the following code snippet:

```
class Foo {
public:
    Foo(Foo&&) noexcept { ... };
};
```

Choose one or more options

- ☒ A move constructor transfers ownership of resources from a temporary object to a new object.
- ☒ The move constructor takes an rvalue reference (`Foo&&`) as its argument.
- ☒ The `noexcept` keyword indicates that the move constructor does not throw exceptions, which is recommended for compatibility with standard containers.
- ☐ The move constructor always performs a deep copy of the source object.
- ☐ The move constructor is called when copying an lvalue to a new object.

Ex. When is the move constructor called? Given the following code snippet:

```
MyClass obj2 = MyClass(obj1);  
std::vector<MyClass> vec;  
vec.push_back(MyClass());  
std::vector<MyClass> vec(10);  
vec.resize(100);  
MyClass obj1;  
MyClass obj2 = std::move(obj1);
```

Choose one or more options

- ☒ The move constructor is called for temporary object initialization (`MyClass obj2 = MyClass(obj1);`).
- ☒ The move constructor is called when pushing a temporary into a container (`vec.push_back(MyClass());`).
- ☒ The move constructor is called during container reallocations if available (`vec.resize(100);`).
- ☒ The move constructor is called when `std::move` is used to force a move (`MyClass obj2 = std::move(obj1);`).
- ☐ The move constructor is never called when returning a local variable from a function.

Ex. Return value and move constructor vs. NRVO. Given the following code snippet:

```
MyClass foo() {  
    MyClass temp;  
    return temp;  
}  
MyClass my = foo();
```

Choose one or more options

- ☒ Scenario 1: The move constructor is called (if available) to transfer `temp` to the return value location.
- ☒ Scenario 2: Named Return Value Optimization (NRVO) allows the compiler to construct `temp` directly in the return value location, avoiding the move constructor call.
- ☐ The copy constructor is always called when returning a local variable, regardless of NRVO.
- ☐ NRVO is illegal in modern C++ and will cause a compilation error.
- ☒ If NRVO is applied, no copy or move constructor is called at all.

Ex. Move constructor automatic generation

Choose one or more options

- ☒ The move constructor is automatically generated by the compiler only if there are no user-declared copy operations, destructor, or move operations.
- ☐ The move constructor is always generated automatically, regardless of other special member functions.
- ☒ The move constructor optimizes resource management by enabling efficient transfer of resources, particularly with resource-heavy objects and standard library containers.
- ☐ The move constructor performs a deep copy of all members of the source object.
- ☒ If the move constructor is not defined and not automatically generated, the copy constructor may be used instead.

Ex. Move assignment operator overview. Given the following code snippet:

```
class Foo {
public:
    Foo& operator=(Foo&& in) noexcept { ... }
};
```

Choose one or more options

- ☒ The move assignment operator transfers resources from a temporary object to an existing object.
- ☒ The move assignment operator takes an rvalue reference (`Foo&&`) and is typically marked `noexcept`.
- ☐ The move assignment operator is used when creating a new object, not when assigning to an existing object.
- ☒ The move assignment operator optimizes performance by avoiding unnecessary copies when assigning temporaries.
- ☐ The move assignment operator always performs a deep copy of the source object.

Ex. When is the move assignment operator called? Given the following code snippet:

```
MyClass obj1, obj2;
obj2 = std::move(obj1);
MyClass obj;
obj = MyClass();
std::vector<MyClass> vec(5);
vec[0] = MyClass();
```

Choose one or more options

- ☒ The move assignment operator is called when using `std::move` to assign one object to another (`obj2 = std::move(obj1);`).
- ☒ The move assignment operator is called when assigning a temporary object (`obj = MyClass();`).
- ☒ The move assignment operator is called when moving a temporary into a container element (`vec[0] = MyClass();`).
- ☐ The move assignment operator is called when creating a new object with `MyClass obj = std::move(obj1);`.
- ☐ The move assignment operator is never called when assigning temporaries; the copy assignment operator is always used instead.

Ex. Move assignment and return values. Given the following code snippet:

```
MyClass createObj() {
    MyClass tmp;
    return tmp;
}
```

Choose one or more options

- ☒ If NRVO (Named Return Value Optimization) is not applied, the move assignment operator may be used when assigning the returned object.
- ☒ If NRVO is applied, `tmp` is constructed directly in the return value location, and no move or copy occurs.
- ☐ The move assignment operator is always called when returning a local object, regardless of NRVO.
- ☐ NRVO is disabled when move semantics are available.
- ☒ The compiler may use the move constructor or move assignment operator to transfer `tmp` if NRVO is not applied.

Ex. Example with copy and move operations. Given the following code snippet:

```
class A {  
    A(); // Primary Constructor  
    A(const A&); // Copy Constructor  
    A(A&&) noexcept; // Move Constructor  
    A& operator=(const A&); // Copy Assignment Operator  
    A& operator=(A&&) noexcept; // Move Assignment Operator  
};  
  
int main() {  
    A a1;  
    A a2 = a1;  
    A a3 = std::move(a1);  
    a3 = a2;  
    a2 = std::move(a3);  
}
```

Choose one or more options

- ☒ A `a1`; calls the primary constructor.
- ☒ A `a2 = a1`; calls the copy constructor.
- ☒ A `a3 = std::move(a1)`; calls the move constructor.
- ☒ `a3 = a2`; calls the copy assignment operator.
- ☒ `a2 = std::move(a3)`; calls the move assignment operator.

Ex. The Rule of Five

Choose one or more options

- ☒ The Rule of Five extends the Rule of Three to include the move constructor and move assignment operator.
- ☒ If a class follows the Rule of Three, it should define all five special member functions in C++11 and later.
- ☐ Not defining move operations leads to incorrect code and runtime errors.
- ☒ Not defining move operations may prevent the compiler from accessing optimization opportunities, reducing performance.
- ☐ The Rule of Five requires defining exactly five member functions, no more and no less.

Ex. Lvalues and rvalues

Choose one or more options

- ☒ Lvalues represent objects that have a name and persist beyond a single expression, with an identifiable memory location.
- ☒ Rvalues represent temporary objects, literals, or values that do not persist beyond the expression and often lack a specific memory location.
- ☐ `int x = 5;` makes 5 an lvalue because it is assigned to x.
- ☒ The result of `x + y` is an rvalue (a temporary object).
- ☒ Move semantics are designed to transfer resources from rvalues, which are about to be destroyed.

Ex. Binding lvalues and rvalues to constructors/operators

Choose one or more options

- ☒ A copy constructor takes an lvalue reference (`const T&`) and binds to both lvalues and rvalues.

- ☒ A move constructor takes an rvalue reference ($T\&\&$) and binds only to rvalues.
- ☒ A copy assignment operator takes a const lvalue reference and binds to lvalues and rvalues.
- ☒ A move assignment operator takes an rvalue reference and binds only to rvalues.
- ☐ An rvalue reference can bind to an lvalue without using `std::move`.

Ex. Function templates overview. Given the following code snippet:

```
template <typename T>
int compare(const T &v1, const T &v2) {
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}
```

Choose one or more options

- ☒ A function template is a formula from which the compiler generates type-specific versions of the function.
- ☒ T is a template parameter representing an arbitrary type.
- ☒ The compiler deduces the type of T from the function arguments when the template is called.
- ☐ The template is processed at runtime to generate the appropriate function version.
- ☒ Templates are the foundation of generic programming in C++.

Ex. Template instantiation. Given the following code snippet:

```
cout << compare(1, 0) << endl;
string s1 = "hello", s2 = "world";
cout << compare(s1, s2) << endl;
vector<int> v1{1, 2, 3}, v2{4, 5, 6};
cout << compare(v1, v2) << endl;
```

Choose one or more options

- ☒ For `compare(1, 0)`, the compiler instantiates `int compare(const int&, const int&)`.
- ☒ For `compare(s1, s2)`, the compiler instantiates `int compare(const string&, const string&)`.
- ☒ For `compare(v1, v2)`, the compiler instantiates `int compare(const vector<int>&, const vector<int>&)`.
- ☐ The compiler generates all possible instantiations of the template at compile time, regardless of usage.
- ☒ The template is never processed at runtime; instantiation occurs at compile time.

Ex. Class templates overview. Given the following code snippet:

```
template <typename T>
class SingleType {
public:
    SingleType(T value) : data(value) {}
    T getData() { return data; }
private:
    T data;
};

int main() {
    SingleType<int> iex(5);
    SingleType<double> dex(3.14);
    return 0;
}
```


}

Choose one or more options

- ☒ A class template is a blueprint for creating classes that work with arbitrary data types.
- ☒ Unlike function templates, the compiler cannot deduce the type T for class templates; it must be specified explicitly.
- ☒ `SingleType<int> iex(5);` creates an instance of `SingleType` specialized for `int`.
- ☐ `SingleType iex(5);` is valid in C++11 because the compiler deduces T from the constructor argument.
- ☒ Class templates enable generic programming for data structures like containers.